

Assignment Questions:

1. Decision Control Statements:

Explain the concept of **decision control statements** in Python. Discuss the syntax and usage of the following:

- `if`, `else`, and `elif` statements
- How these statements are used to execute different blocks of code based on certain conditions. Provide an example where `if`, `else`, and `elif` are used to categorize an integer as positive, negative, or zero.

2. Looping and Nested Loops:

Discuss the difference between **while** loops and **for** loops in Python.

- Describe how both loops work and when to use each.
- Explain the concept of **nested loops** and provide an example of a program that uses a nested loop to print a multiplication table.

3. List, Dictionary, and Set Comprehensions:

Define **comprehensions** in Python.

- Explain **list comprehensions**, **dictionary comprehensions**, and **set comprehensions**.
- Discuss their syntax, how they can be used to create new collections in a concise way, and provide examples to generate:
 - A list of squares of even numbers between 1 and 10
 - A dictionary of square roots of integers from 1 to 5
 - A set containing only unique letters from a string

4. Arrays: 1D and 2D Arrays in Python:

Explain the concept of **arrays** in Python, focusing on:

- How arrays differ from lists in Python
- How to create and manipulate 1D arrays using the `array` module
- How to work with 2D arrays using the **numpy module**
- Provide examples to demonstrate how to create, initialize, and access elements in both 1D and 2D arrays.

5. Functions and Recursion:

Describe the importance of **functions** in Python.

- Explain how functions help in code modularization and reuse.
- Discuss function parameters, return values, and scope of variables within functions.
- Define **recursion** and explain how recursive functions work, providing an example of a function to compute the factorial of a number.

6. Lambda Functions:

Define **lambda functions** in Python.

- Explain how they differ from regular functions defined using the **def** keyword.
- Discuss their syntax, when to use them, and provide an example that demonstrates a lambda function to square a number.

7. Searching Techniques: Linear and Binary Search:

Explain the concepts of **Linear Search** and **Binary Search**.

- Discuss the algorithm and time complexity of both techniques.
- Describe the differences between the two, including when each search method should be used.
- Provide theoretical examples of how each search algorithm works.

8. Sorting Techniques: Selection Sort and Bubble Sort:

Define and explain the **Selection Sort** and **Bubble Sort** algorithms.

- Describe the steps involved in each sorting algorithm.
- Compare the time and space complexities of both algorithms.
- Provide a theoretical example of how each algorithm sorts a given list of integers in ascending order.

Programming Questions:

1. Decision Control Statements:

Write a Python program to determine whether a given number is **prime** or **composite** using **if**, **elif**, and **else** statements. The program should prompt the user to input a number and display whether the number is prime, composite, or neither (e.g., if the number is 1).

2. Looping and Nested Loops:

Write a Python program that prints the **Floyd's Triangle** using nested **for loops**. The program should take an integer **n** as input and print the first **n** rows of the triangle. The triangle should look like this for **n = 5**:

```
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

3. List, Dictionary, and Set Comprehensions:

Write a Python program using **list comprehension**, **dictionary comprehension**, and **set comprehension** to:

- Create a list of all the **even numbers** between 1 and 50.
- Create a dictionary where keys are numbers from 1 to 5 and values are their **squares**.
- Create a set of **unique characters** from the string "programming".

4. Arrays: 1D and 2D Arrays in Python:

Write a Python program that:

- Creates a 1D array using the **array** module and initializes it with integers.
- Creates a 2D array using the **numpy** module and fills it with random numbers between 1 and 100.
- Accesses the element at position (1, 2) in the 2D array and prints it.

5. Functions and Recursion:

Write a Python program that defines a **recursive function** to compute the **factorial** of a given number **n**. Additionally, implement a non-recursive version of the factorial calculation in a separate function.

6. Lambda Functions:

Write a Python program that uses a **lambda function** to:

- Take a list of integers as input from the user.
- Use the lambda function to filter out the even numbers from the list.
- Print the list of even numbers.

7. Searching Techniques: Linear and Binary Search:

Write two separate functions:

- A **linear search** function to find an element in an unsorted list.

- A **binary search** function to find an element in a sorted list. For both functions, the program should display the index of the element if found, or a message saying the element was not found.

8. Sorting Techniques: Selection Sort and Bubble Sort:

Write a Python program that implements both **Selection Sort** and **Bubble Sort** algorithms to sort a given list of integers in **ascending order**. The program should:

- Take a list of integers as input from the user.
- Sort the list using both sorting techniques.
- Print the sorted list for both sorting algorithms.

Additional Notes for the Programming Questions:

- For **decision control statements** and **loops**, focus on using conditional and looping constructs to solve problems that require decision-making and repetitive tasks.
- **Comprehensions** allow concise ways to generate collections like lists, dictionaries, and sets, so encourage the use of comprehensions for concise and readable code.
- For **arrays and numpy**, ensure that students are comfortable with using Python's `array` module and `numpy` library for basic array manipulation.
- **Recursion** and **lambda functions** are advanced topics that focus on problem-solving efficiency and functional programming concepts.
- **Searching and sorting algorithms** help students understand algorithm design, efficiency, and comparison of different methods (linear vs binary search, selection sort vs bubble sort).